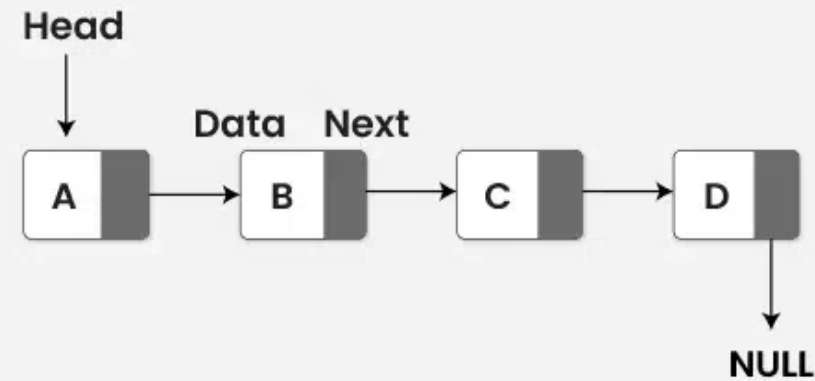


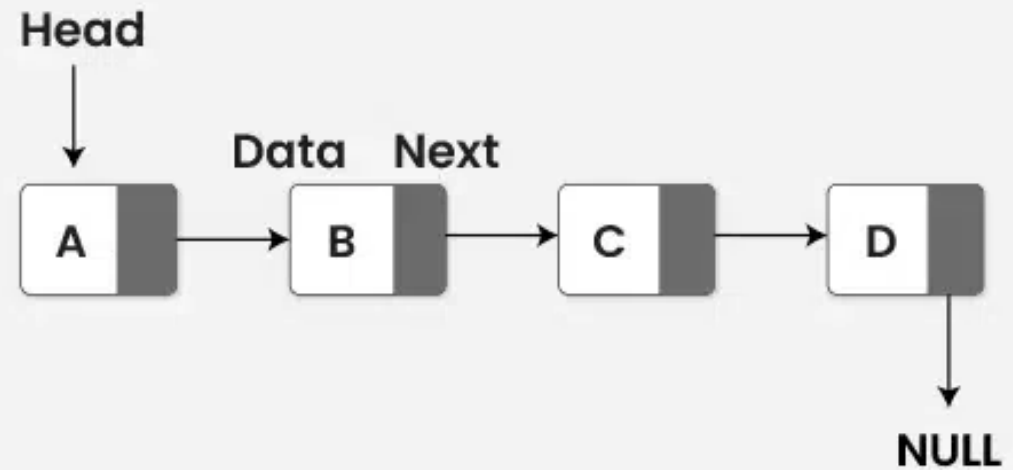
Listas Enlazadas

EIF201 Programación I

Linked List Data Structure



Linked List Data Structure



Listas Enlazadas

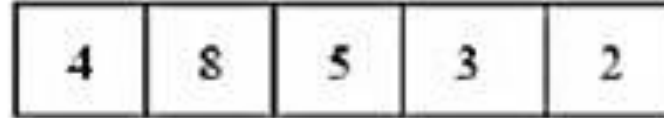
EIF201 Programación I

Lista simplemente enlazada.

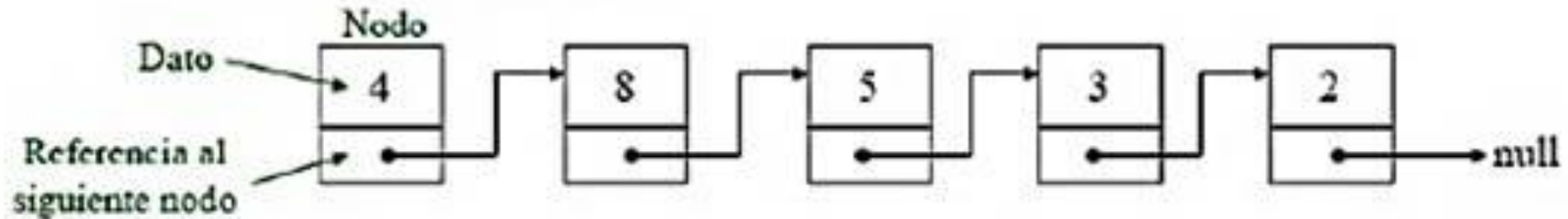


Vector (Secuencial) Vs Lista (Dinámica)

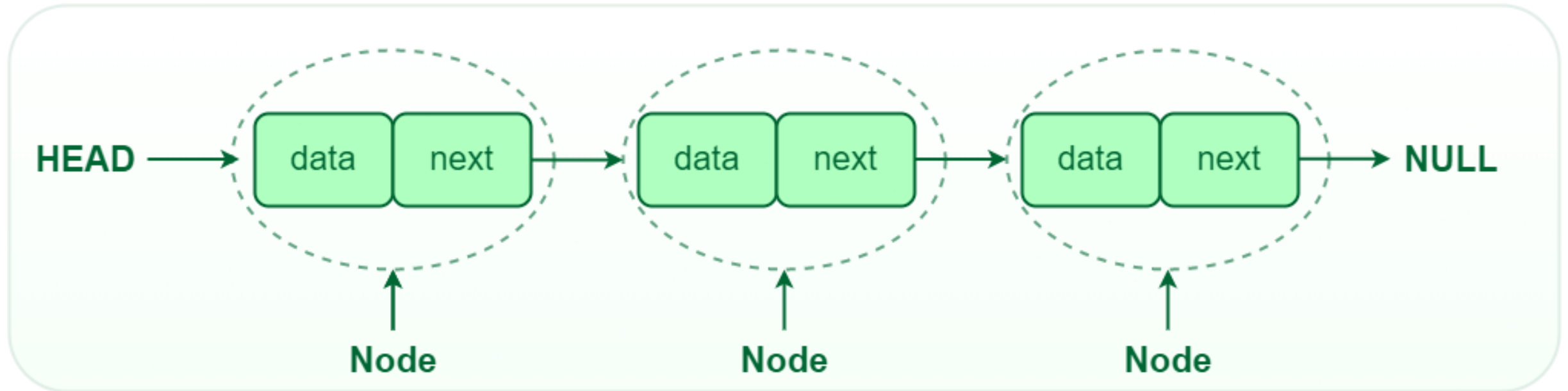
Representación secuencial:



Representación enlazada:

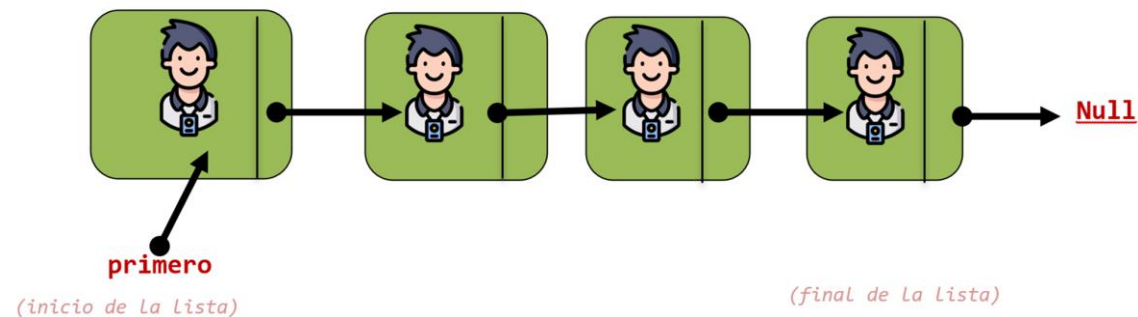


Lista Simplemente Enlazada

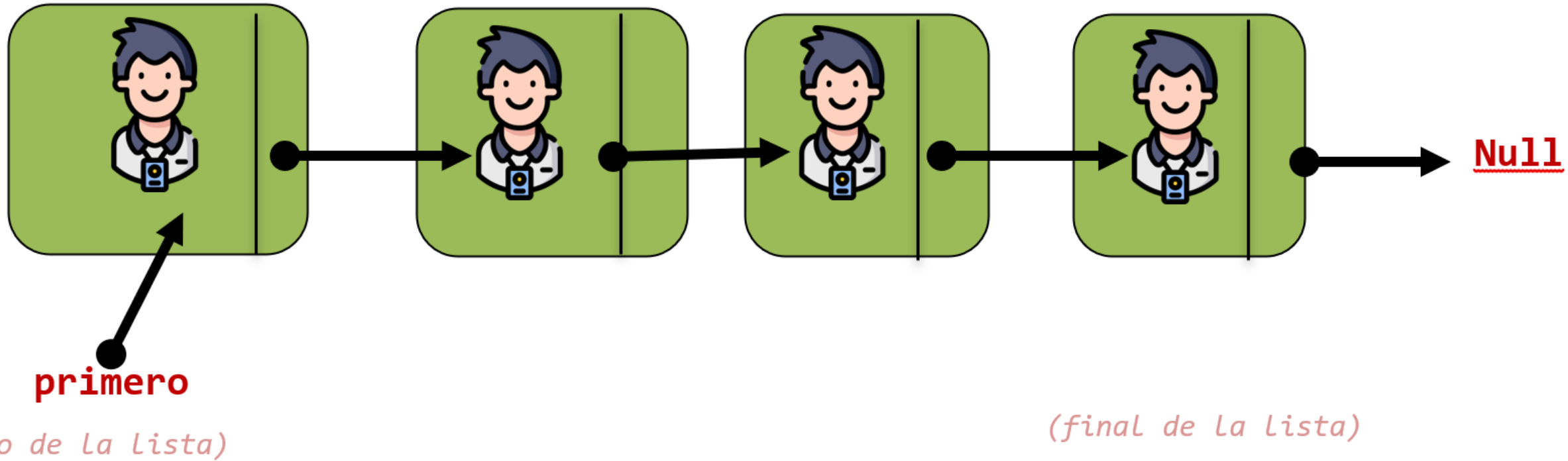


Listas

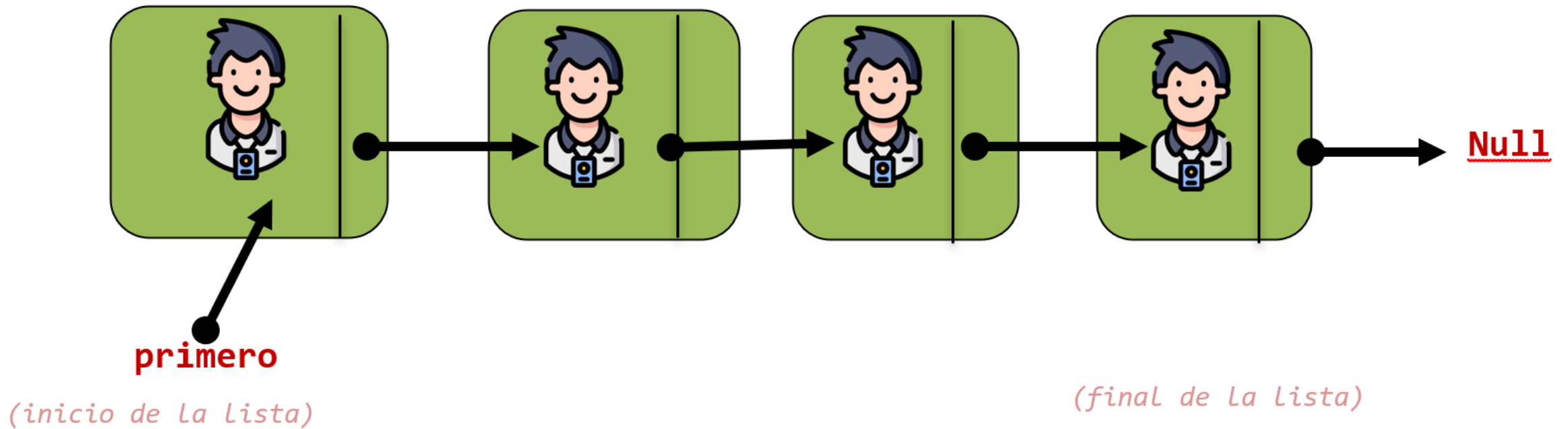
Id	Apellidos	Nombre
402690052	ARROYO ZUMBADO	JUAN
118930104	BONILLA ALVARADO	KARLA
402640350	CAMACHO ALVAREZ	MINOR
208830054	CASTRO SALAS	VICTORIA
119480889	DELGADO KOHKEMPER	MATTIAS
801370002	DELGADO RIVAS	WILLIAMS
119600368	DURAN ESCOBAR	ALONSO
119600367	DURAN ESCOBAR	FERNANDO
119490498	EDWARDS OROZCO	TAYSHAUN
402590518	ESPINOZA ALVARADO	ISAAC
800780187	ESQUIVEL ZABALA	YAOSKA
119560085	GOMEZ SOLIS	SEBASTIAN
402730253	JARA BARTELS	ARNALDO
402710816	JIMENEZ RUGAMA	KENETH
119600374	LOPEZ JUAREZ	BRYAN
402290333	MATAMOROS VARGAS	DIANA
402700683	MENA CHAVERRI	KALEB
119770346	MENJIVAR RAMIREZ	NAARA
402680310	MIRANDA GOMEZ	ALEJANDRO
402690050	OROZCO RAYO	MARIELA
119750839	PEÑA ARANDA ALVARADO	JOSUE
402580173	RODRIGUEZ CHAVES	ANTHONY
119380029	ROJAS BARRANTES	BRYAN
118060056	RUIZ MEDINA	AARON
305670211	SEGURA MIRANDA	KEYLOR
402690459	SOLANO ROJAS	ANDREY
118020323	SOLIS CARVAJAL	ANDREY
119720629	UMAÑA RIOS	WILSON
402650964	VASQUEZ SOLIS	DORIAN
118840531	WAGNER SILVA	EGNIO



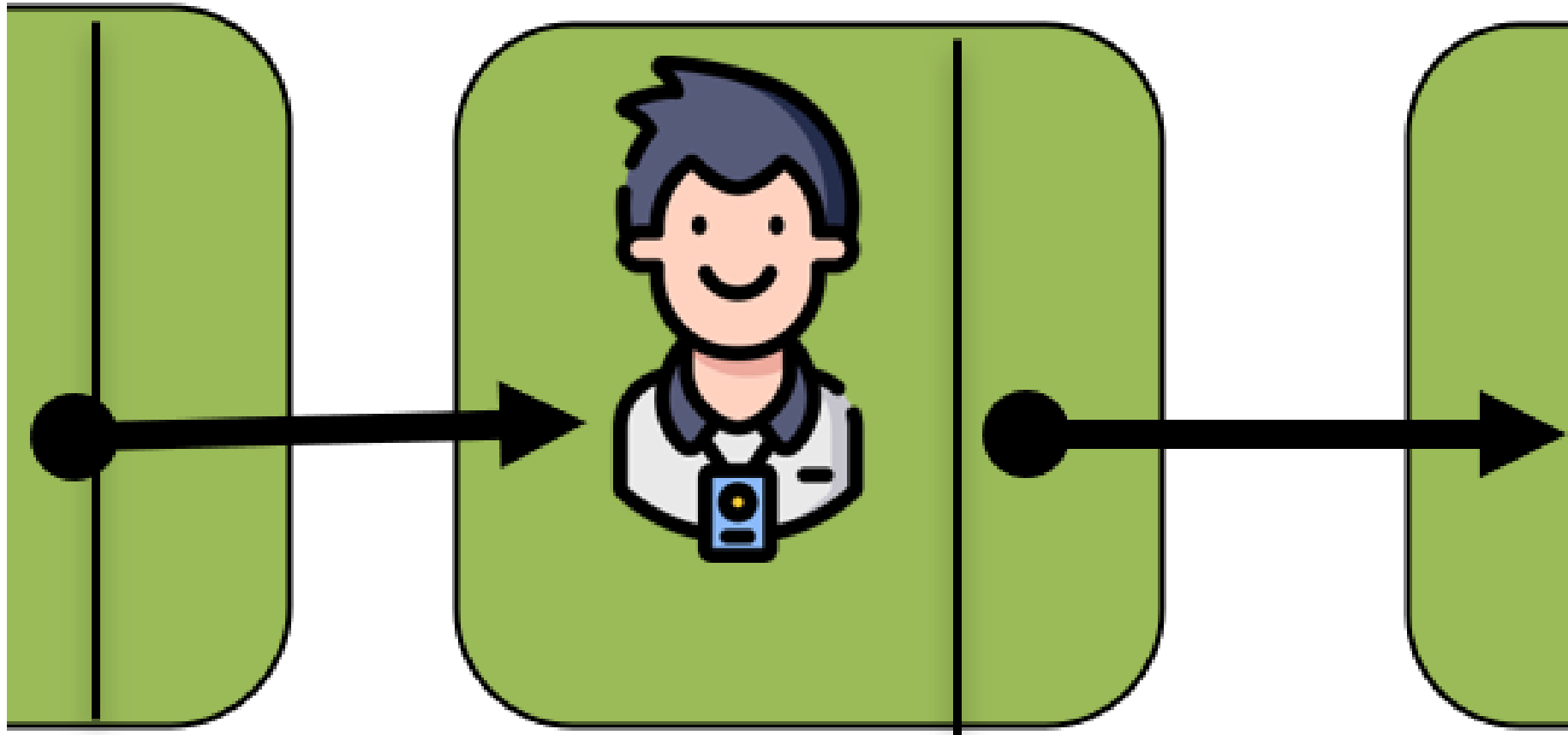
Lista Simplemente Enlazada

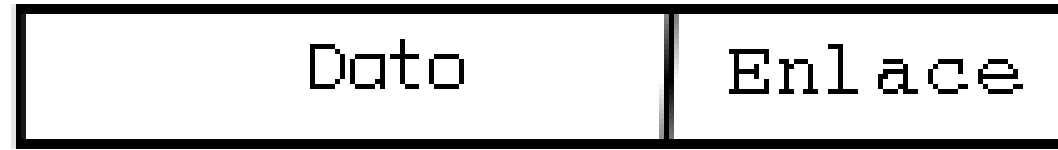


Lista o Cadena de Nodos



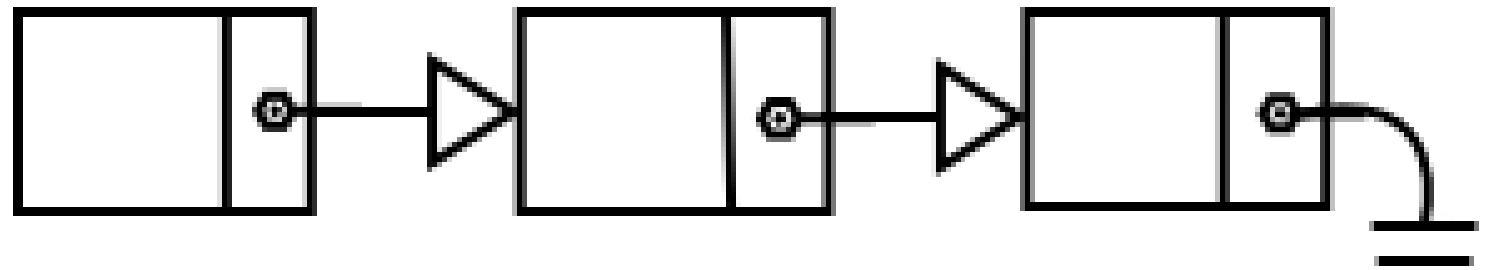
Nodo de la Lista





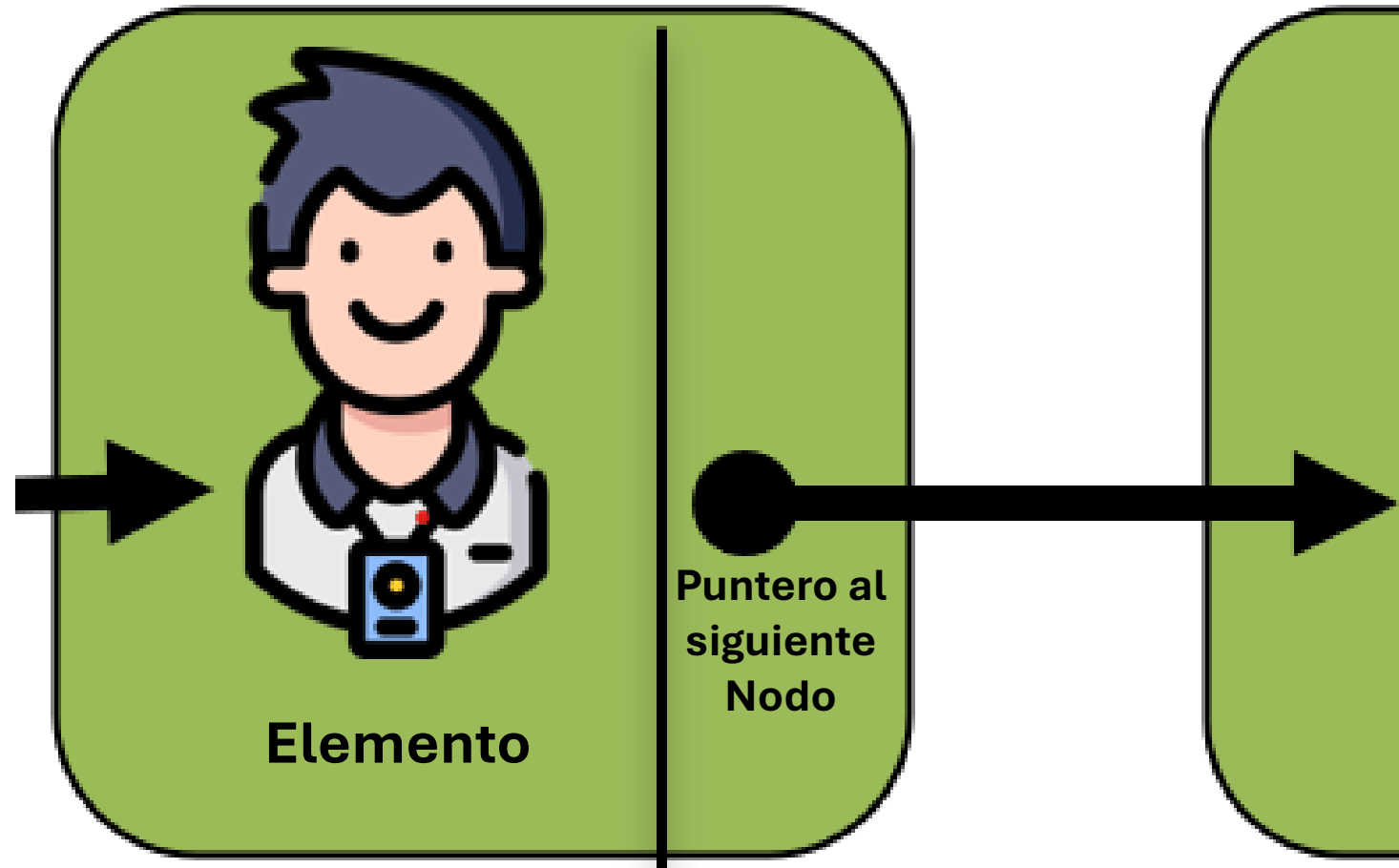
Estructura de un nodo

Nodo de la Lista



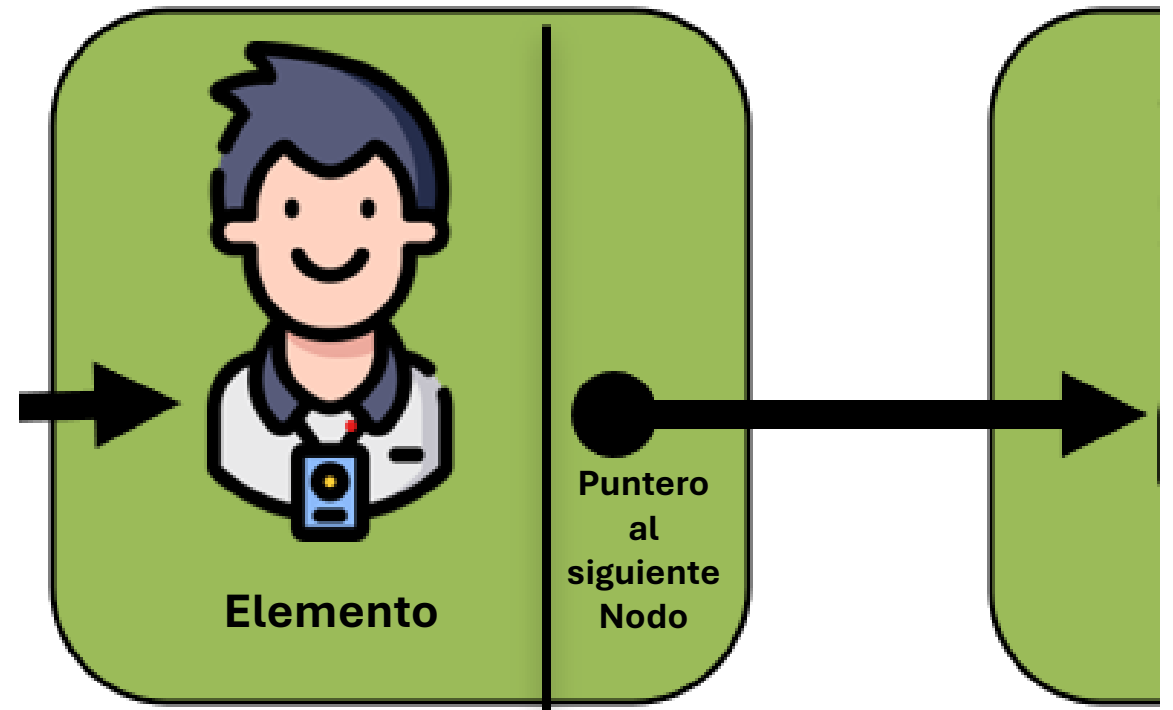
Lista enlazada

Nodo de la Lista

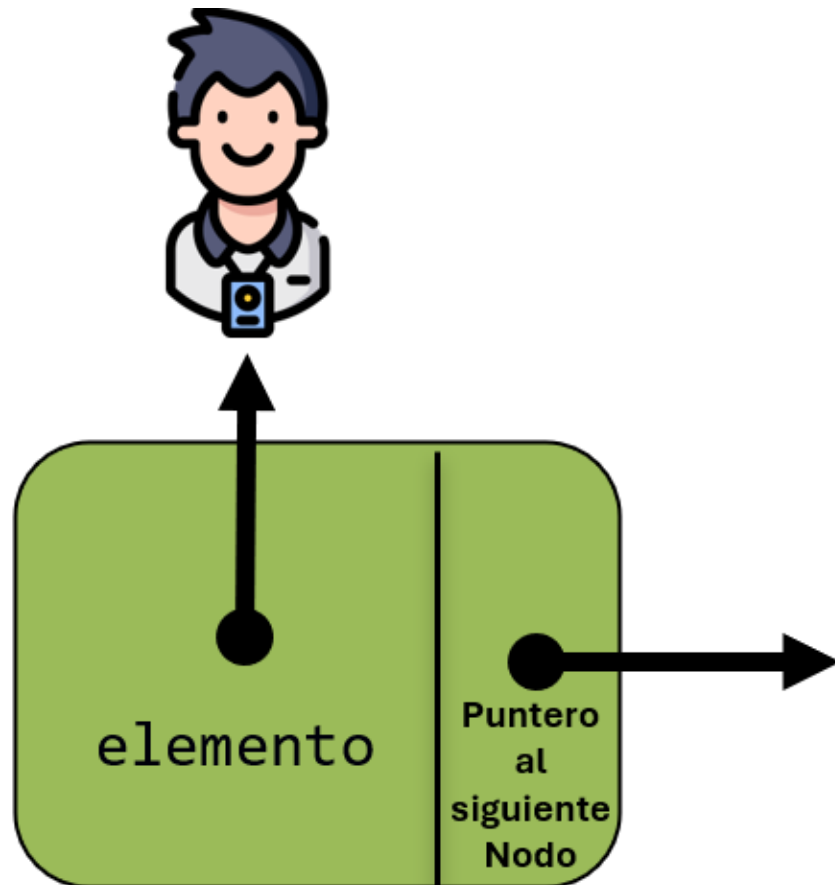


Nodo de una Lista de Objetos

```
class nodo{  
private:  
    persona elemento;  
    nodo * siguiente;  
public:  
    nodo(persona, nodo*);  
    ~nodo();  
    void setElemento(persona);  
    persona getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toStringNodo();  
};
```

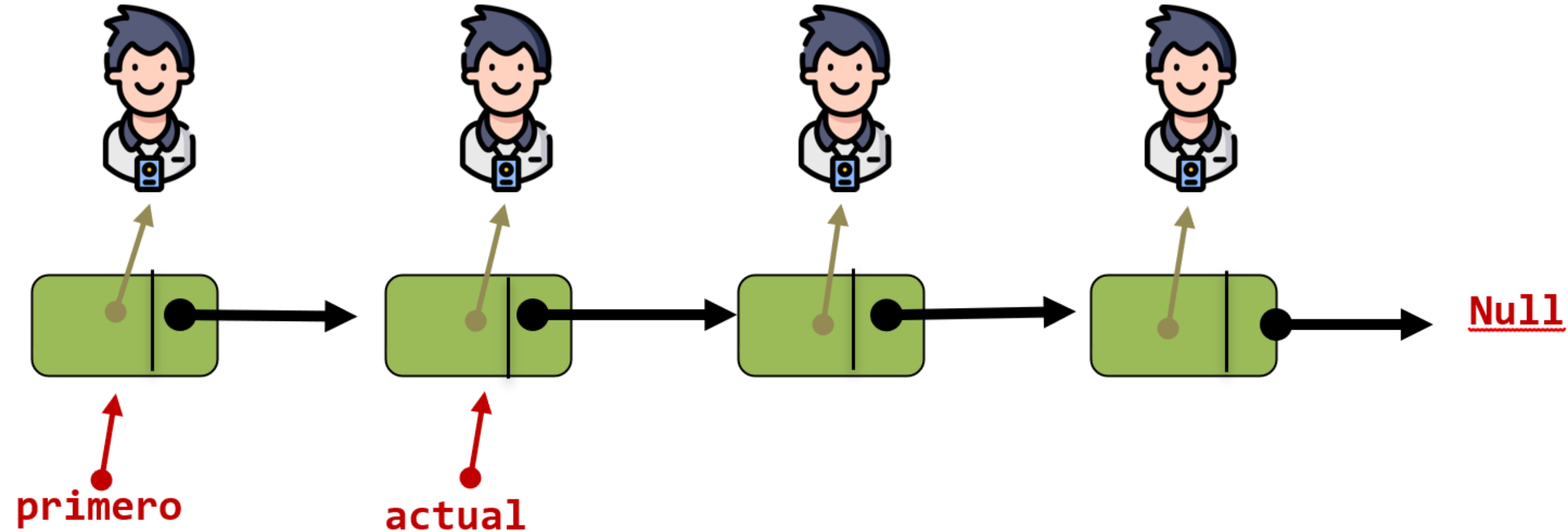


Nodo de una Lista de Punteros a Objetos

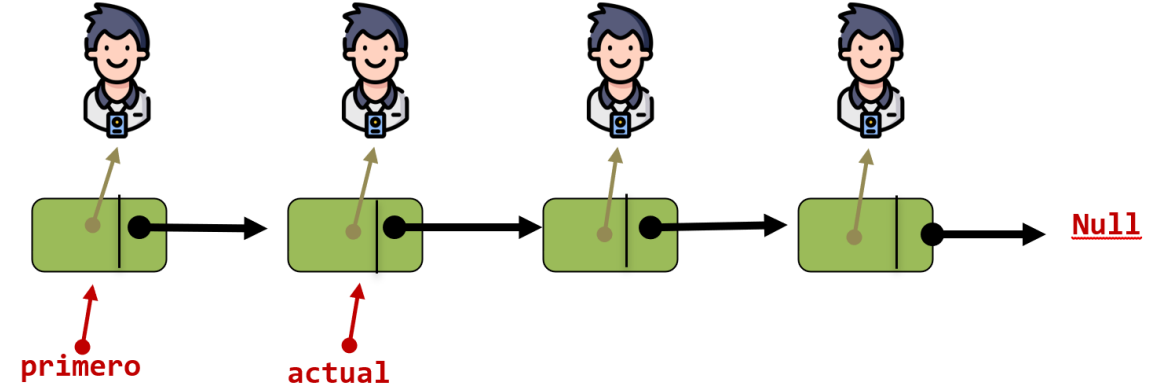


```
class nodo{  
private:  
    persona * elemento;  
    nodo * siguiente;  
public:  
    nodo(persona*, nodo*);  
    ~nodo();  
    void setElemento(persona*);  
    persona* getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toStringNodo();  
};
```

Lista: Colección de Nodos



Lista: Colección de Nodos



```
class lista{
private:
    nodo *primero;    // Puntero al primer nodo de La Lista
    nodo *actual;    // Puntero al nodo de La Lista al que se está accediendo actualmente
public:
    lista(); // La lista siempre empieza vacía (sin elementos).
    void agregar(persona*); // método para agregar un elemento a La lista
    void eliminar(); // método para eliminar un elemento de La lista
    string toString(); // método para mostrar los elementos de La lista
    bool listaVacia(); // método que determina si la lista está vacía
    ~lista(); // destructor de La lista (libera la memoria de los nodos de La lista)
};
```

Lista: Colección de Nodos

```
class lista{
    private:
        nodo *primero;    // Puntero al primer nodo de la lista
        nodo *actual;     // Puntero al nodo de la lista al que se está accediendo actualmente
    public:
        lista(); // la lista siempre empieza vacía (sin elementos).
        void agregar(persona*); // método para agregar un elemento a la lista
        void eliminar(); // método para eliminar un elemento de la lista
        string toString(); // método para mostrar los elementos de la lista
        bool listaVacía(); // método que determina si la lista está vacía
        ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)
};
```

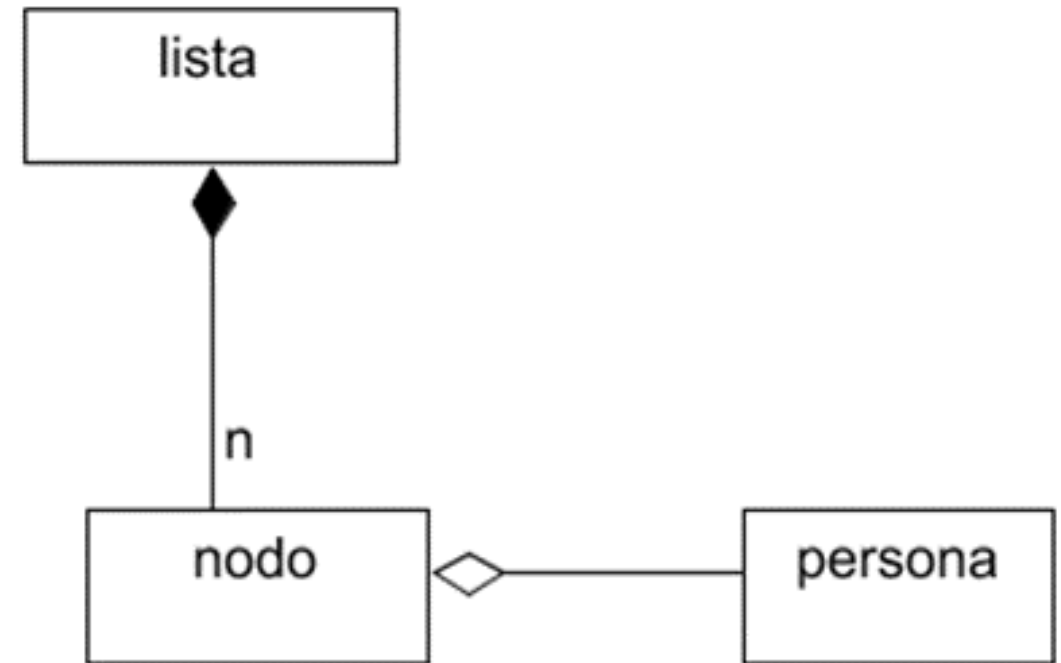
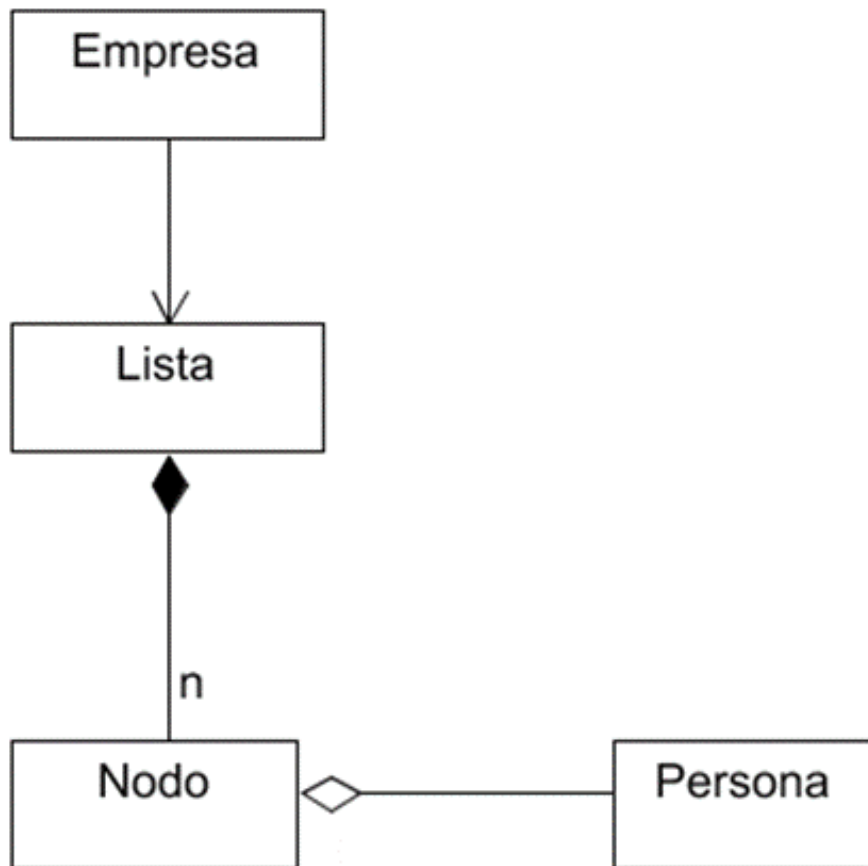

3 Clases: lista, nodo y persona (elemento)



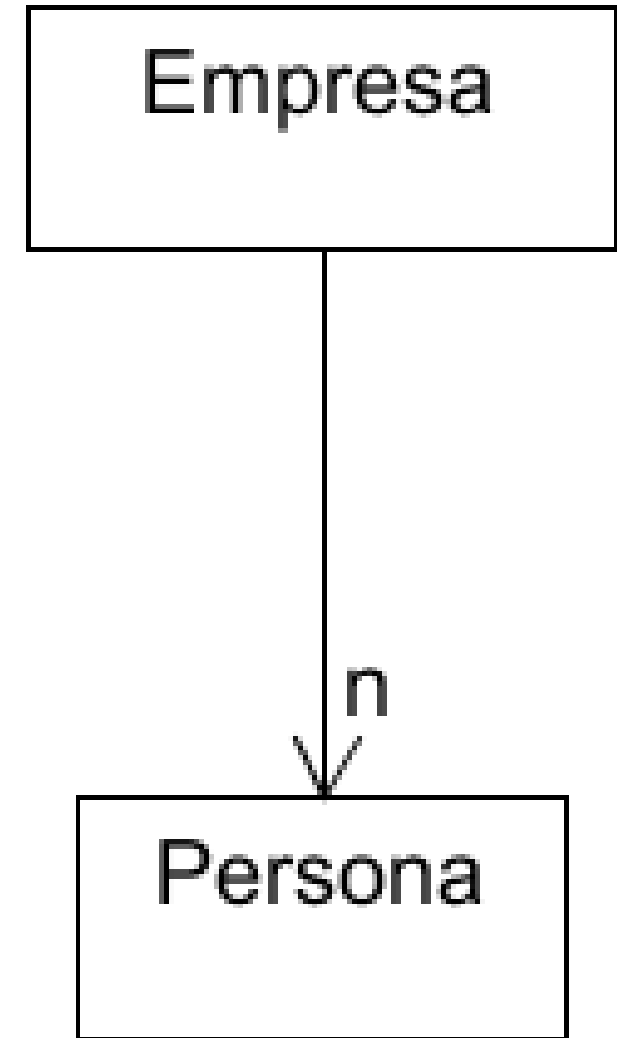
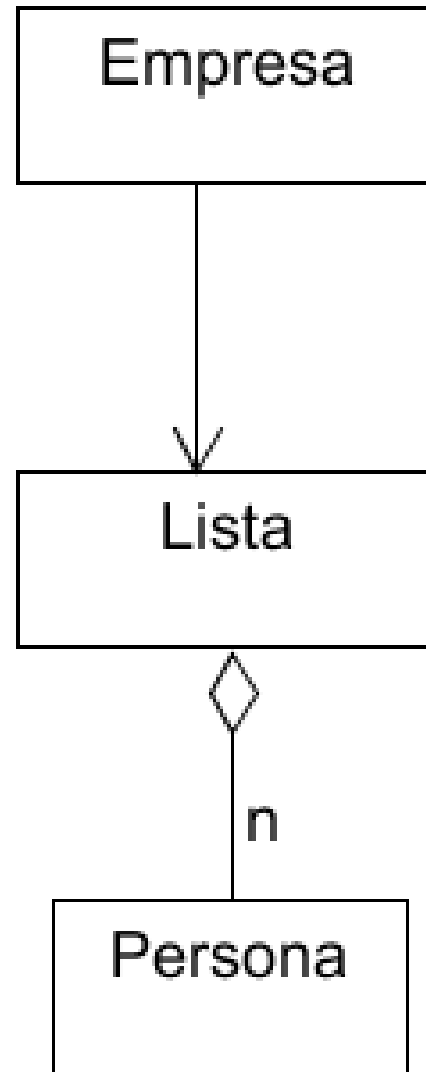
```
class nodo{
private:
    persona * elemento;
    nodo * siguiente;
public:
    nodo(persona*, nodo*);
    ~nodo();
    void setElemento(persona*);
    persona* getElemento();
    void setSiguiente(nodo*);
    nodo* getSiguiente();
    string toStringNodo();
};
```

```
class lista{
private:
    nodo *primero;    // Puntero al primer nodo de La lista
    nodo *actual;    // Puntero al nodo de La lista al que se está accediendo actualmente
public:
    Lista(); // La lista siempre empieza vacía (sin elementos).
    void agregar(persona*); // método para agregar un elemento a La lista
    void eliminar(); // método para eliminar un elemento de La lista
    string toString(); // método para mostrar los elementos de La lista
    bool listaVacía(); // método que determina si la lista está vacía
    ~lista(); // destructor de La lista (libera la memoria de los nodos de La lista)
};
```

Listas Enlazadas: Diagramación UML

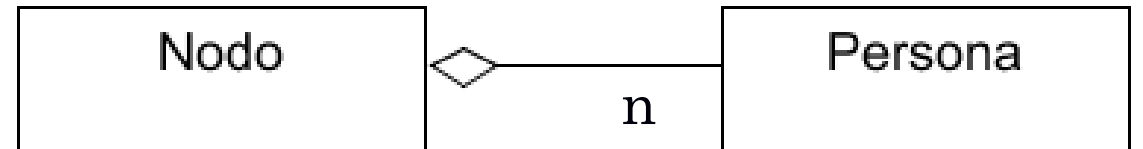
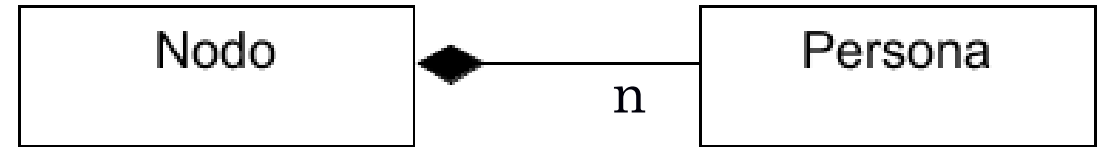


Listas Enlazadas: Diagramación UML



Relación entre las Clases Nodo y Elemento:

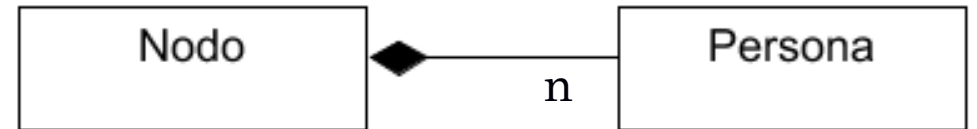
Agregación o Composición



Elementos en Composición

Relación de composición.

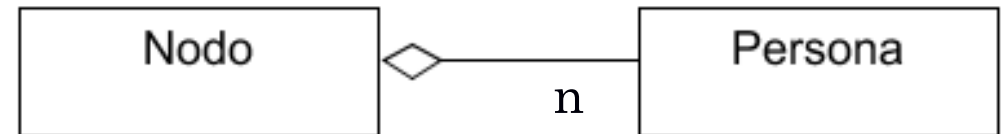
En este tipo de relación, el elemento o “parte” NO puede pertenecer a otros “Todos”. Esto implica que el “todo” debe liberar la memoria de sus “partes”. Por lo tanto, al destruir un nodo, su destructor deberá destruir el elemento o “parte”.



Elementos en Agregación

Relación de Agregación.

En este tipo de relación, el elemento o “parte” Sí puede pertenecer a otros “Todos”. Esto implica que el “todo” NO debe encargarse de liberar la memoria de sus “partes”. Por lo tanto, al destruir un nodo, su destructor NO deberá destruir el elemento o “parte”.



Características de las Listas Enlazadas

- ✓ Son una secuencia “conectada” de nodos.
- ✓ Tienen un nodo inicial (frente o cabeza) y un nodo final (cola).
- ✓ Cada nodo alberga (al menos) dos valores: un valor, elemento o contenido de la lista y un *puntero* o *referencia* que contiene la dirección del nodo que contiene el siguiente valor o elemento de la lista.
- ✓ Para que un nodo pueda acceder al siguiente nodo y la lista no se *rompa*, cada nodo debe tener un puntero con la dirección de memoria que ocupa el siguiente elemento.
- ✓ Una lista solo puede ser recorrida en forma secuencial.
- ✓ Nunca se debe perder la referencia al inicio de la lista. Por lo tanto, se declara un puntero, usualmente llamado primero o inicio, y este debe apuntar siempre al primer nodo de la lista.
- ✓ El ultimo nodo siempre debe tener un puntero a **NULL**.
- ✓ Cuando se aplican restricciones de acceso a las listas, se obtienen pilas y colas, que son listas especiales.
- ✓ Las listas son una estructura de datos más general que las colas y las pilas, por lo que se puede decir que son casos particulares de las listas.

Colas y Pilas

PILA
STACK



LIFO



**ÚLTIMO ELEMENTO EN ENTRAR,
PRIMER ELEMENTO EN SALIR**
(PILA DE PLATOS EN EL RESTAURANTE)

COLA
QUEUE



FIFO

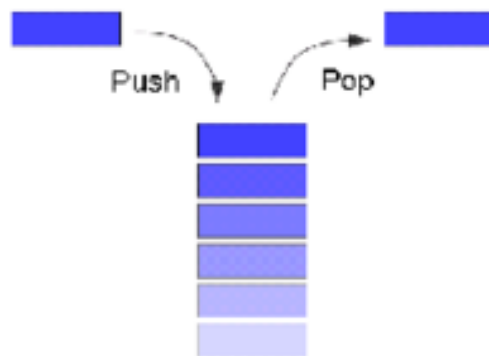


**PRIMER ELEMENTO EN ENTRAR,
PRIMER ELEMENTO EN SALIR**
(COLA EN EL SUPERMERCADO)

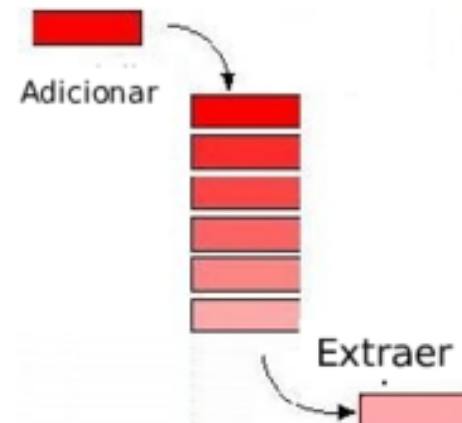
Estructuras Pilas y Colas TDA

Colas y Pilas

Pilas

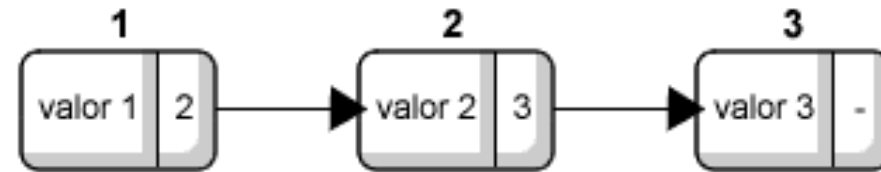


Colas

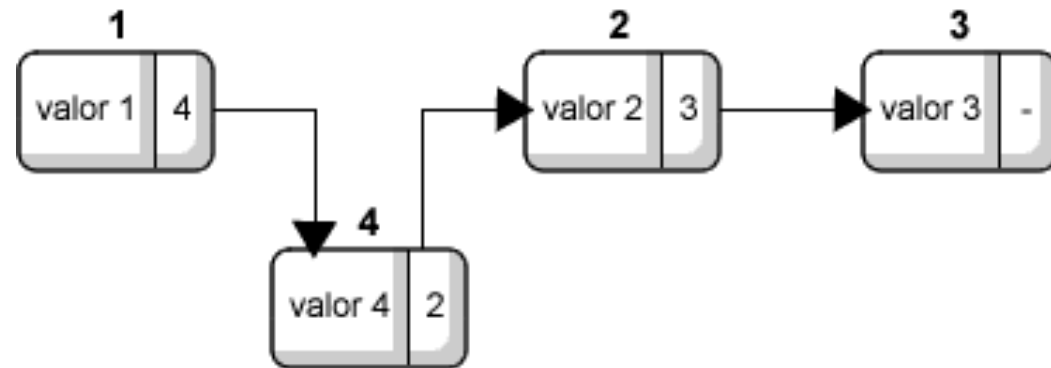


Listas Enlazadas (Operaciones)

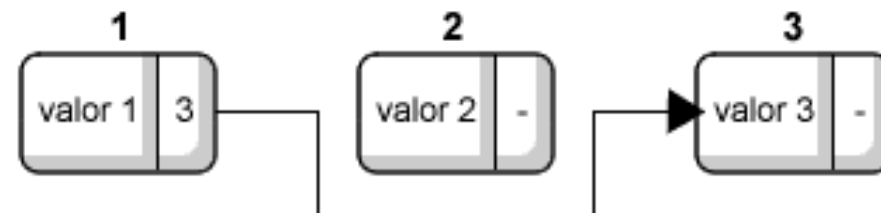
Lista



Agregar



Borrar



Listas Enlazadas (Operaciones)

Lista: Operaciones

- Constructor (Crea una lista vacía)
- Anexar al Final (Anexa un elemento después del ultimo)
- Insertar al Inicio (Inserta un elemento antes del primero)
- Eliminar al Final (Elimina el ultimo elemento)
- Eliminar al Inicio (Elimina el primer elemento)
- Ir al Primero (Se puede acceder al primer elemento)
- Ir al Ultimo (Se puede acceder al ultimo elemento)
- Ir al Siguiente (Se avanza una posición)
- Ir al Anterior (Se retrocede una posición)
- Posicionar (Ubicar el acceso actual sobre el pos-ésimo elemento)
- Información (Retorna el elemento que tiene el acceso actual)
- Longitud (Retorna la longitud de la lista)
- Es fin (Informa si ha llegado al final de la Lista)

3 Clases: lista, nodo y persona (elemento)



```
class nodo{
private:
    persona * elemento;
    nodo * siguiente;
public:
    nodo(persona*, nodo*);
    ~nodo();
    void setElemento(persona*);
    persona* getElemento();
    void setSiguiente(nodo*);
    nodo* getSiguiente();
    string toStringNodo();
};
```

```
class lista{
private:
    nodo *primero;    // Puntero al primer nodo de La lista
    nodo *actual;    // Puntero al nodo de La lista al que se está accediendo actualmente
public:
    Lista(); // La lista siempre empieza vacía (sin elementos).
    void agregar(persona*); // método para agregar un elemento a La lista
    void eliminar(); // método para eliminar un elemento de La lista
    string toString(); // método para mostrar los elementos de La lista
    bool listaVacía(); // método que determina si la lista está vacía
    ~lista(); // destructor de La lista (libera la memoria de los nodos de La lista)
};
```

Crear una lista vacía

```
lista::lista(){  
    primero = NULL;  
    actual = NULL;  
}
```

3 Clases:
lista, nodo y
persona
(elemento)



```
class nodo{  
private:  
    persona * elemento;  
    nodo * siguiente;  
public:  
    nodo(persona*, nodo*);  
    ~nodo();  
    void setElemento(persona*);  
    persona* getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toString();  
};
```

```
class lista{  
private:  
    nodo *primero; // Puntero al primer nodo de la lista  
    nodo *actual; // Puntero al nodo de la lista al que se está accediendo actualmente  
public:  
    lista(); // la lista siempre empieza vacía (sin elementos).  
    void agregar(persona*); // método para agregar un elemento a la lista  
    void eliminar(); // método para eliminar un elemento de la lista  
    string toString(); // método para mostrar los elementos de la lista  
    bool listaVacía(); // método que determina si la lista está vacía  
    ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)  
};
```

Lista simplemente enlazada.



Insertar al inicio de la Lista

```
void lista::insertarInicio(persona p){
    actual = new nodo(p, NULL);
    if (primero == NULL){
        primero = actual;
    }else{
        actual->setSiguiente(primero);
        primero = actual;
    }
}
```

3 Clases:
lista, nodo y
persona
(elemento)



```
class nodo{
private:
    persona * elemento;
    nodo * siguiente;
public:
    nodo(persona*, nodo*);
    ~nodo();
    void setElemento(persona*);
    persona* getElemento();
    void setSiguiente(nodo*);
    nodo* getSiguiente();
    string toStringnodo();
};
```

```
class lista{
private:
    nodo *primero; // Puntero al primer nodo de la lista
    nodo *actual; // Puntero al nodo de la lista al que se está accediendo actualmente
public:
    lista(); // la lista siempre empieza vacía (sin elementos).
    void agregar(persona*); // método para agregar un elemento a la lista
    void eliminar(); // método para eliminar un elemento de la lista
    string toString(); // método para mostrar los elementos de la lista
    bool listaVacía(); // método que determina si la lista está vacía
    ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)
};
```

Lista simplemente enlazada.



Eliminar al inicio de la Lista

```
bool lista::eliminarInicio() {  
    bool posible = primero != NULL;  
    if (posible){  
        actual = primero;  
        primero = primero->getSiguiente();  
        delete actual;  
    }  
    return posible;  
}
```

3 Clases:
lista, nodo y
persona
(elemento)



```
class nodo{  
private:  
    persona * elemento;  
    nodo * siguiente;  
public:  
    nodo(persona*, nodo*);  
    ~nodo();  
    void setElemento(persona*);  
    persona* getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toStringnodo();  
};
```

```
class lista{  
private:  
    nodo *primero; // Puntero al primer nodo de la lista  
    nodo *actual; // Puntero al nodo de la lista al que se está accediendo actualmente  
public:  
    lista(); // la lista siempre empieza vacía (sin elementos).  
    void agregar(persona*); // método para agregar un elemento a la lista  
    void eliminar(); // método para eliminar un elemento de la lista  
    string toString(); // método para mostrar los elementos de la lista  
    bool listaVacía(); // método que determina si la lista está vacía  
    ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)  
};
```

Lista simplemente enlazada.



¿La Lista está vacía?

```
bool lista::vacía() {  
    return primero == NULL;  
}
```

3 Clases:
lista, nodo y
persona
(elemento)



```
class nodo{  
private:  
    persona * elemento;  
    nodo * siguiente;  
public:  
    nodo(persona*, nodo*);  
    ~nodo();  
    void setElemento(persona*);  
    persona* getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toStringnodo();  
};
```

```
class lista{  
private:  
    nodo *primero; // Puntero al primer nodo de la lista  
    nodo *actual; // Puntero al nodo de la lista al que se está accediendo actualmente  
public:  
    lista(); // la lista siempre empieza vacía (sin elementos).  
    void agregar(persona*); // método para agregar un elemento a la lista  
    void eliminar(); // método para eliminar un elemento de la lista  
    string toString(); // método para mostrar los elementos de la lista  
    bool listaVacía(); // método que determina si la lista está vacía  
    ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)  
};
```

Lista simplemente enlazada.



Cantidad de nodos de la Lista

```
int lista::cantidadNodos() {  
    int cantidad = 0;  
    actual = primero;  
    while (actual != NULL){  
        cantidad++;  
        actual = actual->getSiguiente();  
    }  
    return cantidad;  
}
```

3 Clases:
lista, nodo y
persona
(elemento)



```
class nodo{  
private:  
    persona * elemento;  
    nodo * siguiente;  
public:  
    nodo(persona*, nodo*);  
    ~nodo();  
    void setElemento(persona*);  
    persona* getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toStringnodo();  
};
```

```
class lista{  
private:  
    nodo *primero; // Puntero al primer nodo de la lista  
    nodo *actual; // Puntero al nodo de la lista al que se está accediendo actualmente  
public:  
    lista(); // la lista siempre empieza vacía (sin elementos).  
    void agregar(persona*); // método para agregar un elemento a la lista  
    void eliminar(); // método para eliminar un elemento de la lista  
    string toString(); // método para mostrar los elementos de la lista  
    bool listaVacía(); // método que determina si la lista está vacía  
    ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)  
};
```

Lista simplemente enlazada.



Mostrar los elementos de la Lista

```
string lista::toString() {  
    stringstream s;  
    actual = primero;  
    while (actual != NULL){  
        s << actual->getElemento()->toString()<<endl;  
        actual = actual->getSiguiente();  
    }  
    return s.str();  
}
```

3 Clases:
lista, nodo y
persona
(elemento)



```
class nodo{  
private:  
    persona * elemento;  
    nodo * siguiente;  
public:  
    nodo(persona*, nodo*);  
    ~nodo();  
    void setElemento(persona*);  
    persona* getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toStringnodo();  
};
```

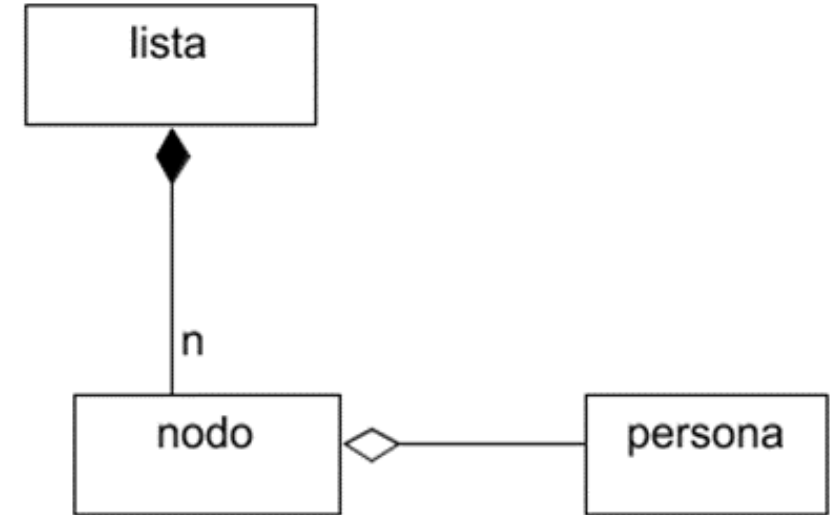
```
class lista{  
private:  
    nodo *primero; // Puntero al primer nodo de la lista  
    nodo *actual; // Puntero al nodo de la lista al que se está accediendo actualmente  
public:  
    lista(); // la lista siempre empieza vacía (sin elementos).  
    void agregar(persona*); // método para agregar un elemento a la lista  
    void eliminar(); // método para eliminar un elemento de la lista  
    string toString(); // método para mostrar los elementos de la lista  
    bool listaVacía(); // método que determina si la lista está vacía  
    ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)  
};
```

Lista simplemente enlazada.



Destruir la Lista

```
lista::~~lista() {  
    while (primero != NULL) { //elimina siempre el primero  
        actual = primero;  
        primero = primero->getSiguiente();  
        delete actual;  
    }  
}
```



3 Clases:
lista, nodo y
persona
(elemento)

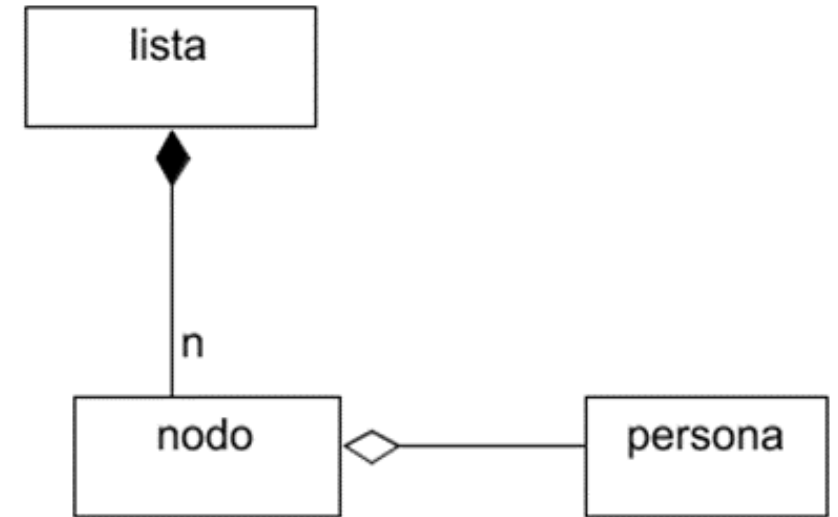


```
class nodo {  
private:  
    persona * elemento;  
    nodo * siguiente;  
public:  
    nodo(persona*, nodo*);  
    ~nodo();  
    void setElemento(persona*);  
    persona* getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toStringNodo();  
};
```

```
class lista {  
private:  
    nodo *primero; // Puntero al primer nodo de la lista  
    nodo *actual; // Puntero al nodo de la lista al que se está accediendo actualmente  
public:  
    lista(); // la lista siempre empieza vacía (sin elementos).  
    void agregar(persona*); // método para agregar un elemento a la lista  
    void eliminar(); // método para eliminar un elemento de la lista  
    string toString(); // método para mostrar los elementos de la lista  
    bool listaVacia(); // método que determina si la lista está vacía  
    ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)  
};
```

Destruir la Lista

```
Lista::~~Lista(){  
    while (eliminaInicio ());  
}
```



3 Clases:
lista, nodo y
persona
(elemento)



```
class nodo{  
private:  
    persona * elemento;  
    nodo * siguiente;  
public:  
    nodo(persona*, nodo*);  
    ~nodo();  
    void setElemento(persona*);  
    persona* getElemento();  
    void setSiguiente(nodo*);  
    nodo* getSiguiente();  
    string toStringNodo();  
};
```

```
class lista{  
private:  
    nodo *primero; // Puntero al primer nodo de la lista  
    nodo *actual; // Puntero al nodo de la lista al que se está accediendo actualmente  
public:  
    lista(); // la lista siempre empieza vacía (sin elementos).  
    void agregar(persona*); // método para agregar un elemento a la lista  
    void eliminar(); // método para eliminar un elemento de la lista  
    string toString(); // método para mostrar los elementos de la lista  
    bool listaVacia(); // método que determina si la lista está vacía  
    ~lista(); // destructor de la lista (libera la memoria de los nodos de la lista)  
};
```

Desafío LSE01

En la clase **lista**, escriba un método **agregarAlFinal(elemento)** que agregue un elemento al final de la lista. Escriba una versión del método para nodos de objetos elemento: **agregarAlFinal(elemento)**, y de nodos de punteros a objetos elemento: **agregarAlFinal(elemento*)**.



Desafío LSE02

En la clase **lista**, escriba un método **eliminarAlFinal()** que elimine el último elemento de la lista. El método debe determinar y devolver si fue posible o no eliminar el elemento.



Desafío LSE03

En la clase **lista**, escriba un método **inicio()** que dirija el puntero **actual** al inicio de la lista.



Desafío LSE04

En la clase **lista**, escriba un método **final()** que dirija el puntero **actual** al último elemento de la lista.



Desafío LSE05

En la clase **lista**, escriba un método **esElFinal()** que determine si el puntero **actual** apunta al último elemento de la lista.



Desafío LSE06

En la clase **lista**, escriba un método **posicionar(int posicion)** que posicione el puntero **actual** en el posicio-ésimo elemento de la lista. El método debe determinar y devolver si fue posible o no posicionar el puntero actual en la posición recibida como parámetro.



Desafío LSE07

Escriba un método **insertar(elemento, posicion)** que permita insertar un elemento en una posición de la lista recibidos como parámetros. Escriba el método en la clase correspondiente. Escriba una versión del método para nodos de objetos elemento: **insertar(elemento, posicion)** y de nodos de punteros a objetos elemento **insertar(elemento*, posicion)**.

Desafío LSE08

Escriba un método **eliminar(int)** que permita eliminar el nodo ubicado en una posición de la lista recibida como parámetro. Escriba el método en la clase correspondiente. El método debe liberar toda la memoria asignada. Escriba una versión del método para nodos de objetos elemento y nodos de punteros a objetos elemento.



Desafío LSE09

Escriba un método **intercambiar (nodo*, nodo*)** que permita intercambiar la posición de dos nodos de la lista. Escriba el método en la clase correspondiente. El método recibe punteros a los nodos anteriores a los que se desean intercambiar. Para referirse al primer nodo se utiliza puntero nulo.



Desafío LSE10

Asumiendo que la clase persona tiene un atributo entero edad y un método `getEdad()`, escriba un método **mayor()** de la clase lista que devuelva un puntero al primer nodo de la lista que contiene o apunta a la persona con mayor edad. Considere que varias personas en la lista pueden tener la misma edad.



Desafío LSE11

Asumiendo que la clase persona tiene un atributo entero edad y un método `getEdad()`, escriba un método **`menorAlInicio()`** de la clase lista que ubique la primera persona de menor edad en el primer nodo de la lista. Considere que varias personas en la lista pueden tener la misma edad.



Listas Enlazadas

EIF201 Programación I

Linked List Data Structure

